Session 3 : PWM and UART

1 Introduction

The informative parts are indicated by the icon , those requiring theoretical reflections are indicated by a are indicated by a symbol , and those requiring programming by the symbol .

1.1 Lab objectives

The purpose of this session is to give you the last tools to tackle the project. We will learn how to control the servo motors using through generation of PWM signals. We will also see how to use serial port communication (UART) to send messages between the PC and the µC while the code is running.
By the end of this session, you should be able to :

- Generate a PWM signal.
- Control the servo motor in real-time.
- Understand the UART serial communication.
- Send data from the PSoC to the PC.
- Send data from the PC to the PSoC.
- Use the Digital-to-Analog Converter (DAC) e.g. by generating a sound on the audio output.
- Make the necessary connections

1.2 Prerequisites

- To know basic programming in C language (variables, arrays, functions, stream control, ect..)
- To have completed Lab 2.
- Here is a link towards PSoC tutorial videos :

(a) Tutorials

FIGURE 1 – Clickable QR code to tutorial videos.

- Here a link towards the wiki.

2 Pulse Width Modulation (PWM)

The purpose of this short assignment is to control the provided servo motor using the potentiometer through a PWM. A PWM is a module that can send a periodic square wave signal such as the one shown in Figure 2.

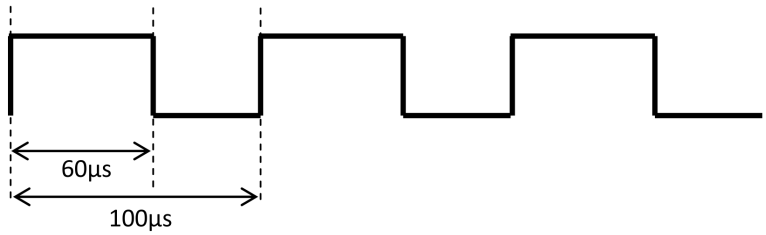


FIGURE 2 – PWM signal with a duty cycle of 60 %.

It is defined by two main parameters :

— The period of the PWM signal i.e. the time duration between two rising edges.

— The duty cycle i.e. the percentage of time the signal is in the high state.

A servo motor is controlled in position i.e. the length of the pulse determines the position of the motor. Here you can find a link to the documentation on PWM component.
The principle of a PWM can be described as being a timer with two thresholds :

1. When the timer starts, the PWM output pin `pwm` is at '1'.
2. When the counting register of the timer reaches the first threshold, named `CMP`, the output `pwm` turns to '0'. The timer continues to increment.
3. When the counting register reaches the value of the period `Period`, the timer returns zero and the output returns to '1'.

You've got it, by playing with the values of `CMP` and `Period`, it is possible to generate any square wave.

Before specifying any value for these registers, we have to know in which range the signal must be in order to control the motor. For this, consult the motor datasheet :

(a) Servo datasheet

FIGURE 3 – Clickable QR code to servo motor datasheet.

Question 1. What are the acceptable values for the time duration of the pulse and the period of the PWM signal ? What are the corresponding values for `CMP` and `Period` ?

Question 2. Which signal should we input on each wire i.e. black, red and orange ?

We are now ready to control the servo motor with a PWM module :

— Instantiate a PWM component in the `TopDesign.cysch`. Set up the registers as required.

> **Hint**
> You realize that the maximum period of a 16-bit PWM module is insufficient for what is required. To increase this range, double-click on the `CLK24MHZ` block in the `TopDesign`. By increasing the divider value, you reduce the rate of the clock driving the PWM.

— Connect a digital output pin to the output of the PWM block.

— Connect the servo to the µC i.e. on the pin you have just configured.

— Run your project.

> **Hint**
> Do not forget to start your component.

Question 3. Why the servo does not move ?

Question 4. Which function would you use to adapt the duration of the pulse within the loop ?

You ask you to do implement this final step :

— Use the potentiometer and ADC code from Lab 1 to make the duration of the pulse vary within the acceptable range depending on the position of the potentiometer.

3 Universal Asynchronous Receiver/Transmitter (UART)

Hint

You will need a usb/µ-usb cable for UART communication.

The purpose of the last assignment is to establish a "real-time" communication between the µC and the PC using UART serial port. The serial bus allows to send data from/to the µC to/from the PC when the code is running on the PSoC. This module is named UART for Universal Asynchronous Receiver/Transmitter. The architecture is presented in Figure 4 where each device has a 2 communication busses on which the symbols transit : a receive (RX) and a transmit (TX) bus. The wiring is done in a crossed way : the TX bus of the first device must be connected to the RX of the other (and vice versa).

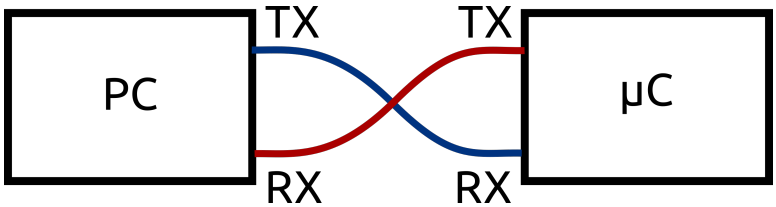


FIGURE 4 – Principle of UART serial port communication between a PC and a µC.

However, the connection shown in Figure 4 cannot be made directly because the PC only has USB ports. For this, we provided you a USB-UART converter i.e. the LC234X. It serves as an interface between the PSoC and the PC and connects through usb/µ-usb cable. Figure 5 shows the final assembly :

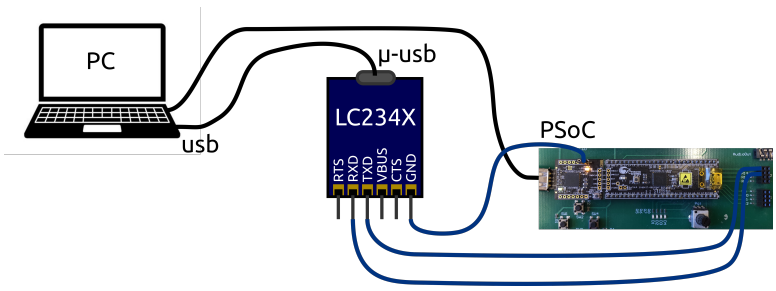


FIGURE 5 – Final assembly for UART serial port communication between the PC and PSoC. Erratum : The GND pin should be connected to the left header not the right one (see Figure 1 in Lab 1 statement).

There are still some configuration steps before you can start the assignment :

1. Make the assembly of Figure 5. Plug the LC234X on the breadboard for that.
2. Download and install the Drivers for the LC234X using this link. This will allow your PC to recognize the usb-UART module.
3. Download and install Termite via this link. It is a shell that allows to make serial communication. It will serve as a starting and ending point for all characters sent from one to the other.

Here are the first instructions on PSoC creator :

- Add an UART component to the `TopDesign.cysch`.
- Connect one digital input and output pins respectively to the RX and TX port of the UART component
- Connect it to the right pins on the `Pins` tab.
- Set the following parameters in the UART component
  - Baudrate : 9600.
  - Data bits : 8.
  - No parity bit.
  - Only one stop bit.
  - No flow control.

Question 5. If you call the function `UART_PutString()` in your firmware, in which direction the data is sent ? From which device to which device ?

3.1 Sending data from PSoC to the PC

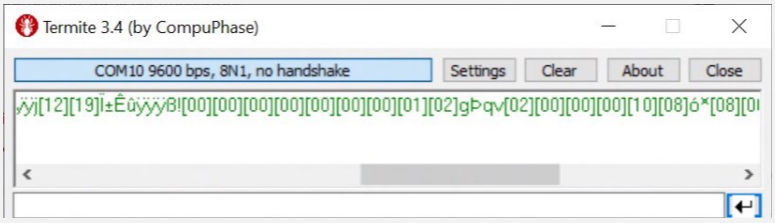
In this short assignment, we will try to send characters and strings from the PSoC to the PC and read them on Termite.

- Check if your usb-UART module is properly recognized by your computer.

  Hint
  Open the Device Manager on your computer << Expand the Ports (COM & LPT) section << Find the COM port associated to the usb-to-serial (or usb-to-UART) converter. To make sure you've identified the right port, unplug and replug the usb/µ-usb cable.

- Make sure the serial port setting on Termite match that of the UART component on PSoC creator (Port, Baudrate,..)

  Hint
  Here is an example of a misconfigured COM port :

- On PSoC Creator, start the UART transceiver.
- Test the UART transmission with the `UART_PutString()` command.
- Send specific data over the UART by using the `UART_PutChar()` command.

Hint

To convert an integer to an ASCII character, you can just add the term '0' (see code snippet below).

```
value_char = value_int + '0';
```

Question 6. Why this code snippet works ?

3.2 Sending data from PC to PSoC

In this last short assignment we want the PSoC to trigger an interrupt each time a byte is received from the PC on the UART port.

To do this,

- Check the "RX - on byte received" box in the UART component in your `TopDesign.cysch` file.
- Add an ISR component connected to the `rx_interrupt`.
- Do not forget to start the interrupt related to the UART reception.
- Try receiving characters from the PC and print them on the LCD by using the code below as starting point.

```
CY_ISR ( isr_uart_rx_Handler ) {
    uint8_t status = 0;
    do{
        // Checks if no UART Rx errors
        status = UART_ReadRxStatus();
        if ((status & UART_RX_STS_PAR_ERROR) |
            (status & UART_RX_STS_STOP_ERROR) |
            (status & UART_RX_STS_BREAK) |
            (status & UART_RX_STS_OVERRUN) ) {
            // Parity, framing, break or overrun error
            LCD_Position(1,0);
            LCD_PrintString("UART err");
        }
        // Check that rx buffer is not empty and get rx data
        if ( (status & UART_RX_STS_FIFO_NOTEMPTY) != 0){

            rxData = UART_ReadRxData();
            // Here comes your code, ...
            // ... what do you do with rxData ?
        }
    }while ((status & UART_RX_STS_FIFO_NOTEMPTY) != 0);
}
```

Question 7. What does this code actually do ?

Question 8. When is an interruption triggered ?

Question 9. How to receive strings from Termite ? How to know when the word is finished ?

> **Hint**
> Check settings in termite << Transmitted text.

Question 10. What happens with longer words ? For instance those with more than 4 characters ?

Question 11. How to store sent words ?